

Natural Language Processing for Data Science (DSA4213)

Doudou Zhou

<https://doudouzhou.github.io/>

Semester 1, AY2026/2027

Class Room: UT-AUD2

Class Room: LT29

Office: S16-6-110

Class Hours: Monday, 2:00 pm to 4:00 pm;

Class Hours: Thursday, 2:00 pm to 4:00 pm;

E-mail: ddzhou@nus.edu.sg

The material below is subject to change. Please stay updated by checking regularly. Last updated on: **2026/08/13**.

Course Description

This course develops Natural Language Processing (NLP) through the intellectual progression that led to modern Large Language Models (LLMs). We begin with the simplest statistical view of language: predicting the next token with an n -gram model. From there, we study distributed word representations, neural language models, recurrent neural networks, LSTMs, sequence-to-sequence learning, attention, and Transformers. This foundation allows us to understand how contemporary LLMs are pretrained, adapted, aligned, augmented with external knowledge, and given additional inference-time computation.

The second half of the course moves from language models that generate text to systems that retrieve information, reason, use tools, maintain memory, interact with other agents, and act within an environment. Topics include instruction tuning, preference optimization, efficient adaptation, retrieval-augmented generation, long-context modeling, reasoning, test-time compute, evaluation, reliability, tool-using agents, multi-agent systems, agent safety, and embodied and multimodal agents. Throughout the course, students will connect mathematical ideas with implementation, experiments, failure analysis, and critical evaluation. The goal is to understand why LLM systems work, when they fail, and what evidence supports their design.

Learning Outcomes

By the end of the course, students should be able to:

1. explain the progression from n -gram language models and distributed representations to RNNs, LSTMs, attention, and Transformers;
2. implement and compare representative language-model architectures using appropriate objectives, decoding methods, and evaluation measures;

3. explain the roles of pretraining, instruction tuning, preference optimization, and parameter-efficient adaptation in developing modern LLMs;
4. design and evaluate retrieval-augmented, long-context, reasoning, and tool-using language-model systems;
5. design, implement, and evaluate a reproducible agent system for a clearly defined task or question;
6. evaluate language-model and agent systems using evidence appropriate to the question, including task-based metrics, human judgment, reliability checks, error analysis, and, where meaningful, baselines or ablations; and
7. identify major limitations and risks involving hallucination, robustness, privacy, prompt injection, tool permissions, and embodied interaction.

Course Expectations and Preparation

This is a hands-on and discussion-intensive module. **Attendance is required.** Students should come to class prepared to ask questions, answer questions, discuss examples, and work through short in-class exercises with peers. At the end of most regular class meetings, there will be a short ungraded exit quiz or practice check. These checks are intended to help students verify their understanding, apply the day’s material, and provide feedback on the pace of the course; they will also be used as part of attendance and participation records. They are distinct from the graded individual quiz/test listed under Assessments. An absence due to documented illness is excused and will not reduce the attendance and participation mark; the student remains responsible for catching up on missed material, coding demonstrations, and exercises.

No prior coursework in NLP or LLMs is required. The course begins with elementary statistical language models and builds the necessary concepts step by step. Students are expected to have prior programming experience in Python, a working knowledge of probability and linear algebra, and a willingness to learn from code, documentation, and technical papers. The individual assignment and project will require substantial independent effort outside class.

Class Format

This module follows an instructor-led narrative in which each new model or system is motivated by the limitations of what came before it. Classes combine concise lectures with guided reasoning, implementation or evaluation activities, and peer discussion. Recurring formats may include:

- **guided predictions**, in which students commit to an expected result before seeing an experiment;
- **mini system labs**, in which students modify or diagnose a small model, retrieval pipeline, or agent;
- **failure clinics**, in which students distinguish an observed failure from its likely cause; and
- **structured discussions**, including preference annotation, evaluation exercises, human-agent simulations, and red-team/blue-team activities.

Tentative Individual Topic Presentations

Beginning around Week 5, students may volunteer to give an individual presentation on a course-related paper, system, failure case, or recent development. Each slot will be no more than 15 minutes, including brief questions, and should explain the central idea, evidence, limitations, and connection to the course. Presentations are tentative and subject to available class time. Topics may be selected from an instructor-provided list or proposed with prior approval.

Assessments

Assessments will include class participation, one individual quiz/test, one individual assignment, and a group project to evaluate your understanding and application of course concepts.

Grading Policy

- Class Participation 5%:
 - Attendance, engagement, questions and responses during class, small-group discussion, constructive feedback during student presentations, and completion of short in-class exit quizzes/practice checks.
- Group Project 45%: Proposal Report (10%); Final Report (25%); Presentation (10%).
- Quizzes/Tests 20%: one individual quiz/test.
- Individual Assignment 30%: one individual assignment.

Important Dates

Dates below are based on the AY2026/2027 Semester 1 calendar and may be adjusted if the university calendar or room schedule changes.

- **Project Proposal**: Due **09/18 at 11:59 pm**.
- **Quiz/Test**: Individual, in-class, closed-book, **10/05**. The scope will be announced in advance.
- **Individual Assignment** (language model comparison, embedding ablation, and downstream representation analysis): Due **10/16 at 11:59 pm**.
- **Final Presentation**: Tentatively **10/29, 11/02, 11/05, 11/12**; 8–10 minutes per group depending on the number of groups.
- **Final Report**: Due **11/13 at 11:59 pm**.

Note: The schedule and dates are subject to change depending on the course progress.

Schedule

- **Week 1 (08/10: Public Holiday; 08/13): Foundations of Language Modeling.**
 1. **Monday: No Class.** National Day public holiday in lieu.
 2. **Thursday: Language Modeling and the Statistical View of Language.** Natural language as data; simple tokenization; sequence modeling; context and dependence; next-token prediction; n -gram language models; maximum likelihood estimation; data sparsity; smoothing and interpolation; toy next-token prediction exercise.
- **Week 2 (08/17, 08/20): Distributed Representations and Recurrent Language Models.**
 1. **Monday: Word Embeddings and Distributed Representations.** From symbolic representations to vector spaces; the distributional hypothesis; Word2vec; negative sampling; co-occurrence methods and GloVe; intrinsic evaluation; ambiguity and limitations; guided embedding-similarity exercise.
 2. **Thursday: Neural Language Models, RNNs, and LSTMs.** Neural language models; recurrent hidden states; training and decoding; vanishing gradients and long-range dependencies; LSTM gates; RNN/LSTM comparison and implementation sketch.
- **Week 3 (08/24, 08/27): Attention and Transformers.**
 1. **Monday: Sequence-to-Sequence Learning and Attention.** Encoder-decoder modeling; neural machine translation as a running example; the fixed-vector bottleneck; attention mechanisms; alignment; limitations of recurrence; guided attention exercise.
 2. **Thursday: Transformer from Attention.** Self-attention; causal masking; multi-head attention; positional encodings; feed-forward layers; residual connections; layer normalization; attention/Transformer notebook walkthrough.
- **Week 4 (08/31, 09/03): Architectures, Tokenization, and Pretraining.**
 1. **Monday: Architecture and Tokenization Choices.** Encoder-only, encoder-decoder, and decoder-only models; subword tokenization; context windows; architectural tradeoffs; how the Transformer resolves limitations encountered in recurrent models.
 2. **Thursday: From Transformers to Foundation Models.** Masked and causal language modeling; encoder, encoder-decoder, and decoder-only pretraining; data mixtures and curation; scaling; synthetic data; deduplication, memorization, and contamination.
- **Week 5 (09/07, 09/10): Instruction Following and Alignment.**
 1. **Monday: From Base Models to Instruction Following.** Prompting; in-context learning; supervised fine-tuning; instruction datasets; task following; evaluation of instruction-tuned models; instructor-led paper/system critique.
 2. **Thursday: Preferences and Alignment.** Preference data; reward modeling; RLHF and RLAIIF; direct preference optimization; human vs. AI feedback; disagreement and alignment tradeoffs; in-class preference-annotation activity; tentative individual topic presentations may begin.

- **Week 6 (09/14, 09/17): Efficient Adaptation and Retrieval Foundations.**
 1. **Monday: Efficient Adaptation and Model Efficiency.** Full fine-tuning vs. parameter-efficient fine-tuning; LoRA/QLoRA; brief comparison with prompt tuning and adapters; practical compute constraints; overview of quantization and distillation.
 2. **Thursday: Retrieval Foundations for Language Models.** Parametric vs. external knowledge; sparse and dense retrieval; embedding-based search; vector indexes; reranking; retrieval metrics; retrieval failure clinic and project proposal clinic.
- **Recess Week (09/21, 09/24): No Class.**
- **Week 7 (09/28, 10/01): RAG, Long Context, and Reasoning.**
 1. **Monday: RAG, Long Context, and Context Engineering.** RAG pipelines; chunking and context construction; source attribution; long-context behavior; RAG vs. fine-tuning vs. long-context prompting; selected extensions such as self-reflective or graph-based retrieval; pipeline walkthrough.
 2. **Thursday: Reasoning and Test-Time Compute.** Chain-of-thought prompting; task decomposition; self-consistency; search over candidate solutions; verifiers; test-time compute; reasoning-oriented post-training; when reasoning traces help, fail, or mislead; accuracy-cost tradeoffs.
- **Week 8 (10/05, 10/08): Evaluation and Reliability.**
 1. **Monday: Quiz/Test and Foundations of LLM Evaluation.** Individual quiz/test; benchmark and task design; automatic metrics; human preference; LLM-as-a-judge; contamination and reproducibility.
 2. **Thursday: Reliability, Robustness, and Error Analysis.** Hallucination; factuality; calibration; robustness; uncertainty; diagnostic evaluation; cost and latency; structured evaluation lab and failure analysis.
- **Week 9 (10/12, 10/15): Agent Foundations.**
 1. **Monday: From Language Models to Acting Agents.** Structured outputs; tool and function calling; observations, actions, and environment feedback; ReAct-style reasoning and acting; agent interfaces; human-agent simulation and simple agent-loop demo.
 2. **Thursday: Planning, Memory, and Long-Horizon Interaction.** State and memory; planning and task decomposition; reflection and recovery; tool selection; long-horizon failure modes; agent debugging exercise; individual assignment due on 10/16.
- **Week 10 (10/19, 10/22): Multi-Agent Systems, Evaluation, and Safety.**
 1. **Monday: Multi-Agent Systems and Agentic Workflows.** Roles, goals, communication, coordination, negotiation, handoffs, and shared world state; centralized vs. decentralized workflows; agentic RAG; Model Context Protocol (MCP); small-group multi-agent simulation.

2. **Thursday: Agent Evaluation, Safety, and Security.** Task success and trajectory evaluation; single-agent and rule-based baselines; memory, retrieval, tool, and agent ablations; cost and latency; prompt injection; instruction hierarchy; privacy; tool permissions; auditability; red-team/blue-team exercise and project clinic.

- **Week 11 (10/26, 10/29): Embodied and Multimodal Agents.**

1. **Monday: From Language Agents to Embodied and Interactive Agents.** Situated intelligence; observations, states, actions, and partial observability; language grounding; perception-planning-action loops; contrastive learning and vision-language models; vision-language-action models; simulated vs. physical environments; embodied evaluation, safety, and open problems.
2. **Thursday: Final Presentations.** Project presentations and discussion.

- **Week 12 (11/02, 11/05): Final Presentations and Peer Discussion.**

1. **Monday: Final Presentations.** Project presentations and discussion.
2. **Thursday: Final Presentations.** Project presentations and feedback.

- **Week 13 (11/09: Public Holiday; 11/12): Final Presentations and Open Problems.**

1. **Monday: No Class.** Deepavali public holiday in lieu.
2. **Thursday: Final Presentations and Open Problems.** Course synthesis; from statistical language models to interactive agents; open problems in NLP, LLM systems, multi-agent interaction, and embodied AI in 2026.

Group Project

The Group Project should address a question related to **agents**. The topic and format are open, provided that the project has a clear objective, a technically meaningful approach, and an appropriate evaluation.

Groups should normally have 4–5 students. Other group sizes require instructor approval so that project scope, workload, and presentation logistics remain manageable.

Individual Contributions

Each final report must include a contribution statement describing each member’s work and giving an approximate contribution share. The shares must sum to 100%. If contributions were broadly comparable, equal shares are appropriate. The group should review the statement together before submission.

If the group cannot agree, or if a student believes that the submitted statement materially misrepresents the contributions, the student may contact the instructor confidentially with specific information or supporting records. Individual project marks may be adjusted from the group mark after considering the contribution statement, confidential reports, version histories, meeting records, and other available evidence. The submitted percentages inform this decision but do not mechanically determine the final marks; the instructor retains final discretion.

Proposal and Scope

The proposal should describe the problem, explain why an agent-based approach is relevant, outline a feasible system or study design, identify the resources required, and provide a preliminary evaluation plan, including comparisons or ablations where they would help answer the question. The proposed architecture may be refined as the course progresses.

Projects should have a focused, feasible scope that supports meaningful evaluation. Groups may use existing models, APIs, datasets, environments, or agent frameworks, but must document them and make a substantive contribution of their own.

Core Requirements

Each project should normally include:

- a clear agent-related problem, question, or hypothesis;
- a well-defined setup describing the agent, task, inputs or observations, outputs or actions, and any relevant tools, knowledge sources, environments, users, or other agents;
- a working implementation or reproducible empirical study of an agent system;
- an evaluation using evidence appropriate to the question, such as task-based metrics, rubrics, human judgments, system measures, case studies, stress tests, or, where meaningful, baselines, comparisons, or ablations;
- analysis of representative successes, failures, limitations, costs, and risks; and
- a reproducible artifact such as code, notebooks, an evaluation suite, a dataset, a prototype, or a documented experimental pipeline.

Possible Directions

Possible directions include:

- a multi-agent virtual world, such as a town, campus, company, or game, used to study cooperation, specialization, information spread, norm formation, or conflict;
- a personal agent that adapts to a user's preferences, routines, writing style, and feedback through memory, retrieval, fine-tuning, or preference learning, with attention to privacy and forgetting;
- a trading agent or team of agents evaluated in simulation or historical backtesting with transaction costs, drawdowns, regime changes, and look-ahead bias handled explicitly;
- tool-using agents for research, coding, data analysis, information seeking, or workflow automation;
- planning, memory, retrieval, reflection, or long-horizon interaction in a single-agent system;
- multi-agent communication, coordination, debate, negotiation, or collaborative problem solving;

- benchmarks and evaluation methods for agent capabilities, reliability, cost, robustness, or safety;
- prompt injection, permission design, privacy, monitoring, and other agent-security questions; and
- human-agent, multimodal, or embodied systems in educational, creative, visual, spatial, interactive, or simulated settings.

Other agent-related ideas are welcome.

Deliverables

The project deliverables should include:

- a proposal and final report describing the question, approach, system or study design, evaluation, findings, and limitations; the final report must also include the required contribution statement;
- runnable code, notebooks, or other project materials with clear setup and documentation;
- experimental results, system outputs, or agent traces as appropriate to the project;
- evaluation evidence and error-analysis results appropriate to the project; and
- a final presentation explaining the project and its main findings.

Evaluation of the Three Project Components

The percentages below are percentages of the overall course grade and sum to the 45% assigned to the Group Project.

- **Proposal Report (10%):** clarity and relevance of the agent-related question; motivation; feasible scope; proposed system or study design; required resources; and a preliminary evaluation plan.
- **Final Report and Project Artifacts (25%):**
 - problem formulation and scope (4%);
 - technical quality and execution (7%);
 - evaluation and evidence (7%);
 - analysis of results, failures, limitations, costs, and risks (4%); and
 - reproducibility, documentation, and responsible practice (3%).
- **Final Presentation (10%):** clarity and organization; quality of the demonstration or examples; communication of the main evidence and limitations; and ability to explain and defend the design and evaluation choices.

Suggested Reading

The readings below are representative starting points. Additional papers, tutorials, and implementation notes may be added during the semester.

- Foundations: language modeling, word representations, attention, and Transformers.
 1. [N-gram Language Models](#) (textbook chapter)
 2. [The Unreasonable Effectiveness of Recurrent Neural Networks](#) (blog post overview)
 3. [Efficient Estimation of Word Representations in Vector Space](#) (original word2vec paper)
 4. [Distributed Representations of Words and Phrases and their Compositionality](#) (negative sampling paper)
 5. [GloVe: Global Vectors for word representation](#)
 6. [Attention Is All You Need](#)
 7. [Jurafsky and Martin Chapter 9 \(The Transformer\)](#)
- Pretraining, post-training, adaptation, and model compression.
 1. [BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding](#)
 2. [Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer \(T5\)](#)
 3. [The Llama 3 Herd of Models](#)
 4. [Training Language Models to Follow Instructions with Human Feedback](#)
 5. [Direct Preference Optimization: Your Language Model is Secretly a Reward Model](#)
 6. [KTO: Model Alignment as Prospect Theoretic Optimization](#)
 7. [LoRA: Low-Rank Adaptation of Large Language Models](#)
 8. [QLoRA: Efficient Finetuning of Quantized LLMs](#)
 9. [Distilling Reasoning Capabilities into Smaller Language Models](#)
- RAG, long context, and knowledge-intensive NLP.
 1. [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#)
 2. [Lost in the Middle: How Language Models Use Long Contexts](#)
 3. [Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection](#)
 4. [From Local to Global: A Graph RAG Approach to Query-Focused Summarization](#)
 5. [Dense Passage Retrieval for Open-Domain Question Answering](#)
- Reasoning and test-time compute.
 1. [Chain-of-Thought Prompting Elicits Reasoning in Large Language Models](#)
 2. [Self-Consistency Improves Chain of Thought Reasoning in Language Models](#)
 3. [Tree of Thoughts: Deliberate Problem Solving with Large Language Models](#)
 4. [DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning](#)
- Tool-using and multi-agent systems.

1. [ReAct: Synergizing Reasoning and Acting in Language Models](#)
 2. [Toolformer: Language Models Can Teach Themselves to Use Tools](#)
 3. [Generative Agents: Interactive Simulacra of Human Behavior](#)
 4. [AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation](#)
 5. [SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering](#)
 6. [Introducing the Model Context Protocol](#)
- Evaluation, reliability, and safety.
 1. [Holistic Evaluation of Language Models](#)
 2. [Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena](#)
 3. [Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference](#)
 4. [SWE-bench: Can Language Models Resolve Real-World GitHub Issues?](#)
 5. [The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions](#)
 - Multimodal and embodied agents.
 1. [Learning Transferable Visual Models From Natural Language Supervision \(CLIP\)](#)
 2. [Visual Instruction Tuning](#)
 3. [Gemini 1.5: Unlocking Multimodal Understanding across Millions of Tokens of Context](#)
 4. [PaLM-E: An Embodied Multimodal Language Model](#)
 5. [Voyager: An Open-Ended Embodied Agent with Large Language Models](#)
 6. [RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control](#)